WhatsApp CRM Architecture Audit Report

Scope: Pre-Redis / pre-BullMQ analysis of the WhatsApp module
Stack: Next.js App Router, TypeScript, MongoDB/Mongoose, Socket.IO, Meta WhatsApp Cloud API, multi-WABA, multi-phone, VPS (single PM2 process)
Date: June 18, 2026
Repository: c:\Admin

Executive Summary

The WhatsApp CRM is a **monolithic single-process architecture**: `socket.ts` runs Next.js, Socket.IO, and two `node-cron` schedulers in one Node process (`pm2` app `adminstro`). Real-time events flow through `src/lib/pusher.ts` (Socket.IO, not Pusher). The inbound webhook (`src/app/api/whatsapp/webhook/route.ts` , ~2,100 lines) is the central event processor.

Primary Performance Bottlenecks

Severity	Issue	Impact
Critical	N+1 unread counting on conversation list (up to ~4 Mongo queries × 25 rows per page)	Slow inbox load
Critical	`notifications/summary` loads all visible conversations + per-conversation `countDocuments`	Slow navbar; scales O(conversations)
Critical	`getEligibleUsersForNotification` scans all WA employees + async access check per inbound message	Webhook latency under load
High	`ConversationArchiveState.find({})` on every inbox request (unbounded)	Full collection scan
High	Per-user socket emits + Expo push in webhook loop	CPU + duplicate client work
High	Synchronous media download/ffmpeg in webhook request path	Blocks event loop
Medium	Triple `countDocuments` on `/conversations/counts` after every list fetch	Extra DB round-trips
Medium	Client joins all phone/channel rooms + 500ms polling interval	Socket overhead

Redis and BullMQ are **not present** today. All caching is in-process `Map` singletons that break under horizontal scaling.

---

1. WhatsApp Architecture Mapping

1.1 API Route Inventory (50 routes)

Webhooks

Route	Methods	Purpose

src/app/api/whatsapp/webhook/route.ts	GET, POST	Meta verification; inbound messages, statuses, calls, history, app-state, echoes

Outbound Messages

Route	Methods
send-message/route.ts	POST
send-media/route.ts	POST
send-template/route.ts	POST
send-interactive/route.ts	POST
send-reaction/route.ts	POST
forward-message/route.ts	POST

Media

Route	Methods
upload-media/route.ts	POST, GET
upload-to-bunny/route.ts	POST

Conversations

Route	Methods
conversations/route.ts	GET, POST
conversations/counts/route.ts	GET
conversations/read/route.ts	POST
conversations/archive/route.ts	POST, DELETE, GET
conversations/transfer/route.ts	POST
conversations/media/route.ts	GET
conversations/[conversationId]/messages/route.ts	GET
conversations/[conversationId]/meta/route.ts	POST
conversations/[conversationId]/labels/route.ts	GET, PATCH
conversations/[conversationId]/preferences/route.ts	GET, PATCH
conversations/[conversationId]/readers/route.ts	GET
conversations/[conversationId]/shared-properties/route.ts	GET

Channels / WABA

Route	Methods
channels/route.ts	GET, POST
channels/[id]/route.ts	PATCH, DELETE
channels/[id]/migrate/route.ts	POST

Templates, Notifications, Calls, Config

Route	Methods
templates/route.ts	GET
notifications/summary/route.ts	GET
notifications/clear/route.ts	POST
call/route.ts	POST, GET
calls/history/route.ts	GET
calls/telemetry/route.ts	POST
ice-servers/route.ts	GET
phone-configs/route.ts	GET
phone-health/route.ts	GET
configured-locations/route.ts	GET
resolve-phone-for-location/route.ts	GET
initiation-limit/route.ts	GET

Search, CRM, Retarget, Leads, Analytics, Admin

Route	Methods
search/unified/route.ts	GET
disposition/route.ts	POST
reminders/route.ts	GET, POST
retarget/route.ts	POST
retarget/upload/route.ts	POST
retarget/uploaded-contacts/route.ts	GET
retarget/handover/route.ts	POST
leads/lookup/route.ts	GET
properties/search/route.ts	GET

`analytics/whatsapp-overview/route.ts`	GET
`admin/migrate-conversation-types/route.ts`	POST
`employee/whatsapp-phone-mask/route.ts`	GET, PUT

1.2 Flow Diagrams

Inbound Message (Webhook)

```
Meta POST /api/whatsapp/webhook
  → Signature validation (HMAC SHA-256)
  → connectDb()
  → processIncomingMessage() per message
      → getActiveChannelByPhoneNumberId          [whatsappchannels]
      → resolveLocationFromLeadPhone             [queries]
      → findOrCreateConversationWithSnapshot     [whatsappconversations, whatsappchannels]
      → getMediaPermanentUrl (if media)          [Meta API → Bunny CDN, ffmpeg for audio]
      → WhatsAppMessage.create / upsert            [whatsappmessages]
      → Automation: guest_questions template     [whatsappmessages, queries, Meta API]
      → WhatsAppConversation.findOneAndUpdate      [whatsappconversations]
      → ConversationArchiveState lookup            [conversationarchivestates]
      → ConversationReadState.find (batch)         [conversationreadstates]
      → getEligibleUsersForNotification          [employees + N×canAccessConversationAsync]
      → Per eligible user:
          → emitWhatsAppEvent(NEW_MESSAGE)         [Socket.IO: user-{id} + phone/channel
rooms]
          → sendIncomingWhatsAppExpoPush             [Expo Push API]
  → Return 200 (always, even on error)
```

Outbound Text Message

```
Client POST /api/whatsapp/send-message
  → getDataFromToken (auth)
  → resolveAllowedPhoneIdsAsync / canAccessConversationAsync
  → resolveOutboundSendCredentials               [whatsappchannels]
  → isWithinMessagingWindow check                [whatsappconversations, whatsappmessages]
  → Meta POST /{phoneNumberId}/messages          [WhatsApp Cloud API]
  → WhatsAppMessage.create                       [whatsappmessages]
  → WhatsAppConversation.update (lastMessage*)   [whatsappconversations]
  → emitWhatsAppEvent(NEW_MESSAGE)               [Socket.IO phone/channel rooms]
  → JSON response
```

Template Send

```
Client POST /api/whatsapp/send-template
  → Auth + access + rental-type template guard
  → findOrCreateConversationWithSnapshot
  → Meta template API call
  → WhatsAppMessage.create
  → RetargetContact update (if retarget flow)    [retargetcontacts]
```

```
  → WhatsAppConversation.update
  → emitWhatsAppEvent(NEW_MESSAGE)
```

Conversation List Load

```
Client GET /api/whatsapp/conversations?limit=25
  → buildInboxListQueryAsync (visibility)        [whatsappchannels, phoneAreaConfigService]
  → ConversationArchiveState.find({})            [ALL archive rows — unbounded]
  → WhatsAppConversation.find + sort + limit    [whatsappconversations]
  → ConversationReadState.find (batch for page) [conversationreadstates]
  → Per conversation (N+1, up to 25×):
      → WhatsAppMessage.findOne (status)
      → WhatsAppMessage.findOne (template inference)
      → WhatsAppMessage.countDocuments (unread)
      → WhatsAppConversation.findByIdAndUpdate (type backfill)
  → Query.find (profile pics)                    [queries]
  → getGuestOutboundStatsByConversationIds       [whatsappmessages aggregate]
  → Employee + "You" conversation logic
  → applyPhoneMaskToConversation
  → JSON (~25 conversation objects)
```

Mark Conversation Read

```
Client POST /api/whatsapp/conversations/read
  → WhatsAppConversation.findById
  → WhatsAppMessage.findOne (latest incoming)
  → ConversationReadState upsert                 [conversationreadstates]
  → emitWhatsAppEvent(conversation-read)
  → io.to(user-{userId}).emit(messages-read)     [navbar only]
```

1.3 Socket.IO Events

Server: `socket.ts` → `(global as any).io`
Emit helpers: `src/lib/pusher.ts`, `src/lib/whatsapp/emitToEligibleUsers.ts`

Event	Direction	Trigger
`whatsapp-new-message`	S→C	Webhook, all send-* routes, createquery, callHistoryService
`whatsapp-message-status`	S→C	Webhook status handler
`whatsapp-message-echo`	S→C	Webhook smb_message_echoes
`whatsapp-new-conversation`	S→C	createquery
`whatsapp-conversation-update`	S→C	Webhook firstReply, archive, meta, mark-first-reply
`whatsapp-conversation-read`	S→C	conversations/read
`whatsapp-messages-read`	S→C	conversations/read (per-user room)
`whatsapp-notifications-cleared`	S→C	notifications/clear

`whatsapp-incoming-call`	S→C	Webhook calls
`whatsapp-call-incoming-offer`	S→C	Webhook USER_INITIATED
`whatsapp-call-missed`	S→C	Webhook
`whatsapp-call-status`	S→C	Webhook
`whatsapp-call-sdp-answer`	S→C	Webhook BUSINESS_INITIATED
`whatsapp-history-sync`	S→C	Webhook history
`whatsapp-app-state-sync`	S→C	Webhook smb_app_state_sync
`retarget_handover`	S→C	retarget/handover (no client listener)

Client joins: `join-whatsapp-room`, `join-whatsapp-phone`, `join-whatsapp-channel`, `join-whatsapp-retarget`
Unused server rooms: `conversation-{id}`, `whatsapp-calls-room`, `whatsapp-sync-room`, `whatsapp-room`

---

2. Database Analysis

2.1 WhatsApp-Native Collections

whatsappconversations

Aspect	Detail
Purpose	Inbox rows; denormalized last-message fields; CRM metadata
Key fields	participantPhone, businessPhoneId, whatsappChannelId, participantLocationKey, lastMessage*, unreadCount, conversationType, isRetarget, leadQueryId, labels, rentalType, channelType
Indexes	Unique {participantPhone, businessPhoneId}; {status, lastMessageTime: -1}; visibility_channel_idx; channel_lookup_idx; crm_labels_inbox_idx
Missing indexes	Text index for search; compound for suffix phone search
Read freq	Very high
Write freq	High

whatsappmessages

Aspect	Detail
Purpose	Full message history
Key fields	conversationId, messageId+businessPhoneId (unique), direction, type, content, status, timestamp
Indexes	{conversationId, timestamp: -1}; {conversationId, direction, status, timestamp: -1}

Missing indexes	{type, reactedToMessageId} for reactions; text index on content.text
Read/Write freq	Very high

whatsappchannels

Purpose: WABA/phone routing, tokens, location assignments. Read: high. Write: low (admin).

whatsappchannelassignments

Purpose: Location→channel routing history.

whatsappcalllogs

Purpose: Call lifecycle / WebRTC telemetry.

whatsappconversationinitiationlogs

Purpose: Daily guest-initiation caps per employee.

conversationreadstates

Purpose: Per-user read cursor. Missing: {userId, lastReadAt: -1} for user-wide unread summary.

conversationarchivestates

Purpose: Global archive. Issue: Full find({}) on inbox load — no pagination.

2.2 Cross-Linked Collections

Collection	WhatsApp usage
queries	Lead lookup, profile pics, disposition, firstReply, visits
employees	Access control, notification recipients, phone mask
retargetcontacts	Retarget upload batches, template send tracking
unregisteredowner / unregisteredownershortterm	Retarget candidate queries
visits	Scheduled from SetVisitDialog via /api/visits/addVisit
personalreminders	whatsappConversationId ref; cron email every 5 min
properties / users	Property search for chat sharing

No dedicated notifications Mongo collection — notifications computed at request time.

---

3. Query Performance Audit

Critical

File	Line	Issue
conversations/route.ts	256–347	N+1: ~75–100 queries per 25-row page

notifications/summary/route.ts	128–165	O(N) conversations; unbounded candidate set
notificationRecipients.ts	21–56	O(employees) on every inbound message webhook
conversations/route.ts	142–144	Unbounded ConversationArchiveState.find({})

High

File	Issue
conversations/archive/route.ts	Same N+1 unread pattern
webhook/route.ts	Per-user socket emit loop
search/unified/route.ts	$lookup with $regex on messages
guestOutboundStats.ts	Aggregate with $regexMatch on message bodies
conversations/route.ts	$regex search on phone/name

Medium

File	Issue
conversations/counts/route.ts	3× countDocuments
messages/route.ts	Extra reaction query; global unreadCount write
phoneAreaConfigService.ts	WhatsappChannel.find({}) on cache miss
analytics/whatsapp-overview	Large aggregations on cold cache

Populate calls: Only 1 in entire WA module (readers/route.ts). No nested populates.

---

4. Conversation Loading Audit

User Opens WhatsApp Dashboard

- **API calls:** phone-configs, call permissions, whatsapp-phone-mask, getLocations (SuperAdmin), conversations?limit=25, conversations/archive (silent), conversations/counts
- **Bottlenecks:** N+1 unread; full archive state load; counts after every list fetch

User Opens Conversation List (filter/search)

- **API:** GET /conversations + counts
- **Bottlenecks:** Regex search; full enrichment pipeline on each filter change

User Opens Single Conversation

- **API:** messages?limit=20, POST /read, GET /templates
- **Bottlenecks:** Meta templates fetch per open; triple parallel on select

User Scrolls Message History

- **API:** messages?limit=20&beforeMessageId=...
- **Bottlenecks:** Low (index present)

User Sends Message

- **API:** POST /send-message (or send-media + upload-to-bunny)
- **External:** Meta Cloud API
- **Socket:** whatsapp-new-message

User Receives Message

- **Path:** Meta webhook → DB → socket (not poll)
- **Bottlenecks:** Synchronous webhook; media download; employee scan; multi-emit

---

5. Socket.IO Audit

Optimization Opportunities

Issue	Recommendation
Duplicate emits	emitToEligibleUsers emits to phone room AND each user-{id}
Per-user webhook loop	Single room emit + client filter
500ms join polling	Event-driven on phone-configs response
Unused rooms	Remove or wire up
retarget_handover	Add client listener
Typing infrastructure	Implement or delete

---

6. Cache Opportunity Analysis

Safe To Cache (Long TTL)

Channel config, phone→location map, Meta templates, CRM labels, ICE credentials.

Cache With Short TTL

Conversation list page, counts, phone health, analytics, search results, guest outbound stats.

Must Never Be Cached

Access tokens, real-time message content, webhook idempotency keys, initiation limits, call SDP, phone mask rules.

---

7. BullMQ Opportunity Analysis

Immediate Queue Candidates

Operation	Queue	Benefit
Webhook processing	wa-webhook-inbound	Fast 200 ACK
Media download + Bunny	wa-media-process	Unblock event loop
Expo push	wa-push-notify	Faster webhook

Notify fanout	wa-notify-fanout	Parallelize emits
guest_questions automation	wa-automation	Isolate failures

Should Never Be Queued

User-initiated send, mark read, call signaling, auth checks.

---

8. Redis Readiness Report

In-Memory / Global State

Location	Redis migration
socket.ts global.io	Socket.IO Redis adapter
webhook lastEmitMap	SET NX PX 300 debounce
webhook pushedForMessageUser	SET EX 86400 idempotency
phoneAreaConfigService cache	Redis hash
phone-health, analytics, search caches	Redis with TTL

Race Conditions

Multi-instance without Redis adapter; per-process debounce; unreadCount vs ConversationReadState inconsistency; cron ×2 if scaled horizontally.

---

9. VPS Infrastructure Audit

Item	Value
Host	VPS /var/www/adminstro
CI	.github/workflows/deploy.yml
PM2	1 instance, tsx socket.ts
Port	3000
Redis	None
Workers	None

CPU Hotspots

Webhook ffmpeg, send-media ffmpeg, inbox N+1, notifications/summary, unified search, analytics cold miss.

Worker Separation Candidates

Webhook + media worker, cron scheduler, Socket.IO server, BullMQ workers, analytics precompute.

---

10. Final Architecture Recommendation

Phase 1 — Query Fixes + Redis Foundation (2–4 weeks)

1. Fix N+1 in GET /conversations (aggregation-based unread)
2. Fix notifications/summary (limit + aggregate + Redis cache)
3. Optimize getEligibleUsersForNotification (cache per visibility tuple)
4. Deploy Redis (Socket adapter, debounce, shared caches)

Estimated gain: Inbox 2–5s → 300–800ms
Complexity: Medium

Phase 2 — BullMQ + Webhook Worker (3–5 weeks)

1. Webhook fast-ACK → enqueue → 200
2. Queues: wa-webhook-inbound, wa-media-process, wa-notify
3. Separate PM2: adminstro, adminstro-worker, adminstro-cron
4. Job dedup by wamid

Estimated gain: Webhook p99 < 100ms; 40–60% lower web CPU under bursts
Complexity: High

Phase 3 — Scale + Advanced Caching (4–8 weeks)

1. Redis unread counter service
2. Inbox list cache with socket invalidation
3. Text index or Meilisearch for message search
4. Horizontal scale with sticky LB + Redis adapter
5. Analytics precompute via BullMQ

Estimated gain: Sub-200ms inbox; 3–5× user load
Complexity: High

Microservices Assessment

Not required now. Consider dedicated notification/media service only if >100k messages/day sustained or webhook processing routinely exceeds 5s.

Appendix: Key File Reference

Concern	Path
Webhook	src/app/api/whatsapp/webhook/route.ts
Inbox API	src/app/api/whatsapp/conversations/route.ts
Messages API	src/app/api/whatsapp/conversations/[conversationId]/messages/route.ts
Visibility filters	src/lib/whatsapp/inboxQuery.ts, locationAccess.ts
Socket emit	src/lib/pusher.ts, emitToEligibleUsers.ts
Notification recipients	src/lib/whatsapp/notificationRecipients.ts
Main UI	src/app/whatsapp/whatsapp.tsx

Socket server	socket.ts
Deploy	.github/workflows/deploy.yml

Report generated for senior architect review. Analysis only — no code changes.